



Tecnológico de Monterrey

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS
SUPERIORES DE MONTERREY

CAPSTONE PROJECT (TEAM #49)

Key Findings

Author Julia Gabriela Pinedo A01795315

Professor Dr. Grettel Barceló Alonso

Project Advisor Dr. Guillermo Mota Medina

November 2, 2025

Contents

1	Model Analysis	3
1.1	Results	3
1.2	Performance	4
2	Actionables	6
2.1	Recommendations	6
2.2	Next Steps	6
3	Implementation	8
3.1	Cloud Implementation	8
4	References	11

1 Model Analysis

1.1 Results

As established in **Stage 0**, the project had the following objectives:

- Establish a single-agent reinforcement learning baseline.
- Extend the same baseline algorithm to a multi-agent setting.
- Design and implement a robust multi-agent Reinforcement Learning framework that enables effective cooperation among multiple differential-drive robots in simulated environments.
- Integrate a custom differential-drive robot model equipped with odometry, IMU, and contact detection sensors to support a realistic RL environment in Gazebo + ROS 2.
- Evaluate performance across key metrics (success rate, efficiency, robustness under agent failures, and adaptability to obstacles).

During the **implementation phase**, we successfully created a simulation environment in Gazebo Harmonic integrated with ROS 2 Jazzy, where a custom differential-drive robot model was developed and equipped with odometry, IMU, and contact detection sensors; with the purpose of enabling a realistic state feedback for the Reinforcement Learning process (**Figure 1.1**).

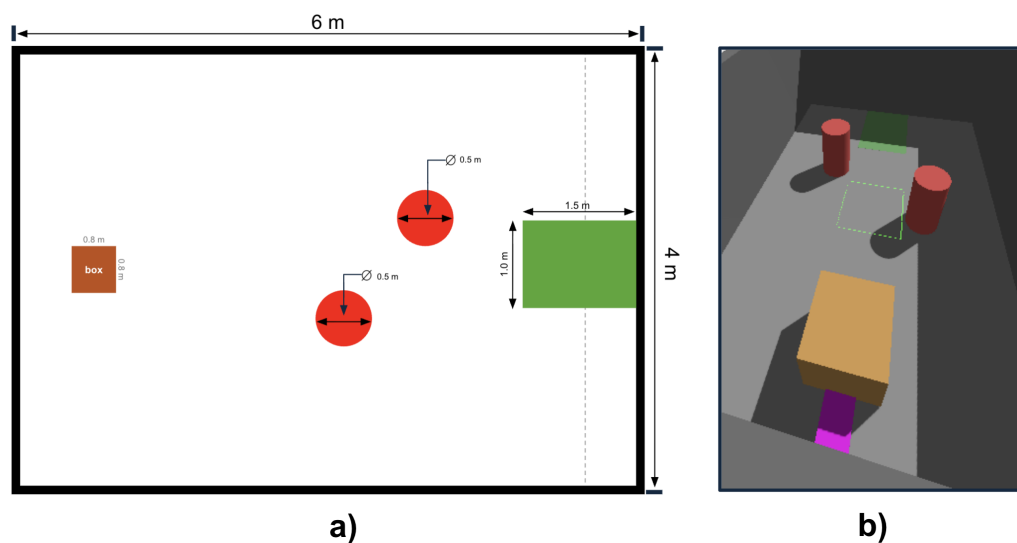


Figure 1.1 Simulation environment. a) Diagram of the arena, b) Arena modeled and simulated in Gazebo.

This simulation is designed to be **scalable and modular**, allowing adaptation between single-agent and multi-agent configurations for easy control experiments and prediction. This flexibility ensures that the environment is able to support future cooperative learning scenarios without requiring a complete redesign.

However, due to the time constraints of the current stage, the full implementation of the multi-agent RL framework (*including communication, coordination strategies, and shared policy optimization*) will be addressed as part of the **next development phase**.

1.2 Performance

To evaluate the performance of the model, we established the following core evaluation metrics:

- **Episode return over time (learning curve):**

This measures whether the policy improves its ability to maximize cumulative rewards, reflecting how effectively the robot learns to push the box toward the goal as training progresses.

- **Success rate (goal completion):**

Captures the fraction of episodes in which the box reaches the goal region, serving as a direct and interpretable indicator of task achievement independent of reward shaping.

- **Episode length/steps to success (efficiency):**

Evaluates how quickly the robot completes successful episodes, with shorter trajectories indicating higher operational efficiency.

The selected **floor criteria** have the *eval_success_rate* (evaluation success rate) as the principal metric, since it measures task completion under evaluation conditions and remains robust to variations in reward shaping (**Table 1.1**).

Table 1.1 Floor criteria.

Metric
Eval success rate ≥ 0.20
Median steps-to-success ≤ 80
Positive learning trend on eval (rolling/local slope > 0)
No overfitting signal (<i>eval_return</i> – <i>train_return</i> at eval points) ≥ -1.0

The following results (**Table 1.2**) were obtained for the selected final model: Advantage Actor-Critic (A2C) with Bagging.

Table 1.2 Performance of final model: Advantage Actor-Critic + Bagging.

Metric	Result
Eval success rate ≥ 0.20	0.79
Median steps-to-success ≤ 80	56 steps
Positive learning trend on eval (rolling/local slope > 0)	0.067
No overfitting signal ($eval_return - train_return$ at eval points) ≥ -1.0	0.127

To visualize the performance of the model on the selected metrics and summarize the learning dynamics of the final model, the following graphs were generated (**Figure 1.2**):

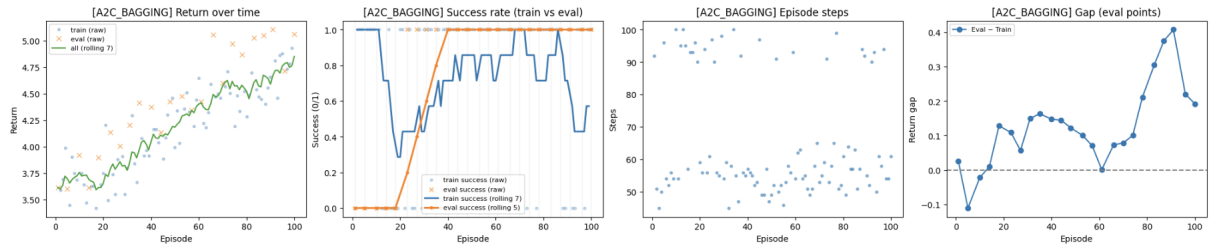


Figure 1.2 Graphs of the A2C Bagging model.

We observe that the performance of the model **meets the floor criteria**, making it strong enough to serve as a reference policy. This meets objectives 1, 4, and 5 and provides the standard for objectives 2 and 3, which expect the expansion into a multi-agent setting. Since the model is stable, reproducible, and reliable, it is acceptable to be **production-ready**, where it can serve as the reference model for the next phase and be reused as a foundation to train, compare, or extend into the multi-agent framework.

In terms of the **margin for improving the performance of the model**, some refinements can still be applied to strengthen the baseline without modifying the simulation environment, for example:

- Including stronger penalties where errors are most costly could encourage a smoother and more purposeful behavior.
- Implementing a tighter *exploration-to-exploitation* schedule, for example, by gradually reducing the entropy bonus, would allow the agent to shift from broad exploration toward more consistent, goal-directed actions as training progresses.

These adjustments would enhance the stability and efficiency of the model, providing the ideal groundwork before scaling the system to the multi-agent setup, as coordinated behavior will be critical to achieve a robust cooperative performance.

2 Actionables

2.1 Recommendations

Beyond exploration tweaks, the following recommendations were established to fully implement the solution:

- 1. Standardization of the computational environment.** Although the use of Windows Subsystem for Linux has supported initial experiments, migrating to a system with Ubuntu Linux 22.04 for the simulation is recommended for best compatibility. In addition, shipping a single Dockerfile (*CPU + optional CUDA tag*) will guarantee that the runs are reproducible on any machine.
- 2. Automate and share the analysis pipeline.** To make the result visualization transparent and reusable, a Jupyter Notebook should be created to import metrics, generate the plots automatically, the summary table, and save the output figures to ingest them into reports or LaTeX documents for easy tracking.
- 3. Tighten and stabilize the single-agent baseline.** As discussed previously, applying stronger penalties for inefficient or unstable actions and adopting a tighter exploration-to-exploitation schedule will improve training stability. These refinements would also provide a cleaner and more deterministic reference for the future multi-agent cooperation experiments.

2.2 Next Steps

According to **Stage 0**, the project involves a small team composed of Dr. Fuxi Wen (Academic Stakeholder) and Julia Gabriela Pinedo (Master's Student), who will have the following tasks:

Academic Stakeholder

- Define success gates for the multi-agent and the single-agent simulations comparison. For example, the multi-agent has to exceed the single-agent by +10-15% success rate and demonstrate equal or better efficiency.
- Review and validate each iteration. Once they meet the expectations, approve moving from the single-agent to the multi-agent setup.
- Define the fixed maps, seeds, and run budgets to keep the comparisons fair and consistent.

Master's Student

- Refine the baseline with the characteristics previously defined (stronger penalties, exploitation implementation).
- Create a Jupyter Notebook to load metrics, generate the 4 graphs + summary DataFrame, and export the figures for reports or LaTeX documents.
- Prepare the environment and documentation to facilitate the transition to the multi-agent framework.

3 Implementation

3.1 Cloud Implementation

Since the project’s main purpose is to establish a strong single-agent benchmark as the foundation for a future multi-agent setup, the plan is to develop these policies on real robots eventually. In preparation for that, the single-agent benchmark will be packaged into a **dockerized training/evaluation harness** to ensure portability and easier adaptation in subsequent implementations.

For Cloud evaluation, the following platforms were prioritized: Google Cloud (Vertex AI), AWS (SageMaker), Microsoft Azure (Azure ML), and IBM (watsonx.ai). The following figures (**Figure 3.1-3.2**) provide a comparison between the platforms. [1][2][3][4]

Criterion	Google Cloud	AWS	Azure	IBM
Ease of use	Simple custom or prebuilt containers. Clear job submission. Well-documented SDK. Good Jupyter integration.	Mature SDK. Powerful pipelines but a steeper learning curve for new users.	Solid SDK. Tightly integrated with Azure AD.	Focused on foundation model governance. Custom RL setups require more configuration.
Scalability	Preemptible VMs and managed jobs simplify large evals.	Flexible instance zoo.	Solid autoscale on endpoints. Large hardware ecosystem.	Oriented to enterprise. Pricing tied to services used.
Services offered	Vertex AI Training. Workbench (Jupyter). Prediction (batch/online). Model registry. Cloud logging/monitoring.	SageMaker Training/Processing. Endpoints. Model registry. Debugger/Profiler. Studio/Studio Lab.	Azure ML Jobs. Model registry. MLflow integration. AI dashboard. Azure monitor.	Watson studio. Auto AI. Notebooks/SDK deployment spaces. Custom RL simulations.
Associated costs (price per month, assuming 120 CPU hours + 40 GPU T4 hours)	64 dollars/month (plus storage/network).	59 dollars/month (plus storage/data transfer).	86 dollars/month (plus disks/network)	1,050 dollars/month (automatic plan amount for a month)

Figure 3.1 Description of the targeted platforms.

Services	Google Cloud	AWS	Azure	IBM
Managed custom training (containers)	First-class custom/prebuilt containers. Simple job submit. Good for Dockerized trainers.	Mature training and pipelines. Strong registry and lineage.	SDK/CLI for training. Pipelines and endpoints are part of it.	Studio with model/train + governance. Fewer options for custom RL sim workflows.
Endpoints for serving/ dashboards	Vertex Prediction for batch.	Real-time batch endpoints and inference pipelines.	Online and batch endpoints with network isolation.	SDK and services exist. Emphasis on FM/gov tooling.
MLOps	Unified platform for experiments and registry.	Pipelines and model registry are advanced.	Full lifecycle management with strong enterprise controls.	Strong governance. MA/RL simulation less opinionated.
Simulation relevance	Works well for Gazebo and ROS2.	Robomaker is discontinued, so we need to create a custom pipeline.	Need for creating a custom pipeline.	Need for creating a custom pipeline.
Portability to real robots	Clean edge/on-prem migration.	Clean and large hardware ecosystem.	Strong for hybrid/on-prem.	Possible, but geared to large enterprise processes.

Figure 3.2 Description of the services from the targeted platforms.

We selected **Google Cloud (Vertex AI)** as the platform for implementation because of its moderate cost for light, non-24/7 simulation workloads and its container-first workflow, which supports custom training, reproducible pipelines, and straightforward batch evaluations. Vertex AI also integrates seamlessly with Jupyter and other Python-based tools, making it the best choice to maintain the same environment used locally during experimentation [1]. This ensures that the trained models can be later containerized, versioned, and reproduced consistently.

As stated in the previous section, the selection is temporary, since the experiments and reporting will run on a local computer, while the final deployment will be done on real robots using Tsinghua University’s in-house infrastructure, which already supports reinforcement-

learning rollouts. Choosing Vertex AI at this stage allows for scalable yet cost-efficient experimentation, while maintaining compatibility with the containerized workflows that will be transferred to the university's infrastructure for real-robot implementation.

4 References

- [1] “Vertex ai custom training overview. ”[Online]. Available: <https://cloud.google.com/vertex-ai/docs/training/overview>.
- [2] “Pipelines steps - amazon sagemaker ai. ”[Online]. Available: <https://docs.aws.amazon.com/sagemaker/latest/dg/build-and-manage-steps.html?>.
- [3] “Azure machine learning documentation. ”[Online]. Available: <https://learn.microsoft.com/en-us/azure/machine-learning/?>.
- [4] “Ibm software hub. ”[Online]. Available: <https://www.ibm.com/docs/en/software-hub/5.2.x>.